

AI-DLC & the Multi-Repo Workspace

A complete guide: the problem, the architecture, the methodology, and the day-to-day workflow.

Ecommerce Platform Workspace · AI-DLC = **AI-Driven Development Life Cycle** · Based on [aws-labs/aidlc-workflows](https://aws-labs.github.io/aidlc-workflows)

How to put this into Google Docs (keep the formatting)

1. Open this `.html` file in Chrome (double-click it).
2. Select everything: **Cmd/Ctrl + A**, then copy: **Cmd/Ctrl + C**.
3. In your Google Doc, paste: **Cmd/Ctrl + V**. Headings, tables, colours, and diagrams carry through.

Note: pasted images can occasionally need a manual re-drag in Google Docs. The diagrams are built as tables, so they always survive the paste.

Contents

1. The problem we are solving
2. Solution, part 1 — the workspace + git submodules
3. Working with submodules (the how-to)
4. Solution, part 2 — what AI-DLC is and why
5. The complete AI-DLC flow (the three phases)
6. How you interact with it — the gate model
7. The core discipline — never vibe code
8. How AI-DLC is wired into this repo
9. AI-DLC in your IDE (screenshots)
10. Glossary & quick reference

1. The Problem We Are Solving

There are actually **two** distinct problems, and this workspace solves both. It helps to keep them separate in your head.

Problem A — AI loses context across repositories

When a system is split across many repos, AI coding tools lose the big picture. By default, if you open a single service repo, the AI only knows about *that* service. It doesn't know what other services exist, how they talk to each other, what contracts they share, or what decisions have already been made. The results:

- The AI suggests changes that quietly **break other services**.
- Logic gets **duplicated** because the AI doesn't know it already exists elsewhere.
- API shapes drift **inconsistent** because the AI can't see the shared contracts.
- Refactors miss **inter-service dependencies** entirely.

The naive fix — keeping a separate "context repo" next to the service repos — doesn't work. You can't guarantee everyone cloned it, put it at the right path, or kept it in sync. The AI still won't automatically pick it up when you open a service folder.

Problem B — AI development without discipline ("vibe coding")

Even with full context, letting an AI generate code from a one-line prompt produces work with **no plan, no record of decisions, and no checkpoints**. Requirements are assumed rather than confirmed, design choices are invisible, and nobody can later answer "why was it built this way?" This is fast on day one and expensive every day after.

The two-part answer. A **workspace repo with git submodules** solves Problem A (context). The **AI-DLC methodology** solves Problem B (discipline). The rest of this document explains each, and how they fit together in this repo.

2. Solution, Part 1 — The Workspace + Git Submodules

We use a single **workspace repo** that everyone clones. It does not contain service code directly — it stitches the independent service repos together as **git submodules**. Open the workspace root in your IDE (or run Claude Code from it) and the AI can see every service at once. **One clone, full context.**

Each service repo stays independent — its own git history, its own CI/CD, its own deployments. The workspace just records a *pointer* (a commit hash) to a known-good version of each service.

The repo structure

```
workspace/ (everyone clones THIS – the parent repo)
├── CLAUDE.md           ← global AI context + cross-service rules
├── .cursor/rules/      ← same context, for Cursor
├── aidlc-workflows/    ← submodule → awslabs/aidlc-workflows (EXTERNAL, not ours)
├── aidlc-docs/         ← AI-DLC working artifacts (tracked in git)
├── shared-contracts/   ← source of truth for API specs + event schemas
│   ├── api-specs/
│   └── events/
└── services/          ← service repos as submodules
    ├── user-service/   ← submodule → ArfaMujahid/user-service
    ├── order-service/  ← submodule → ArfaMujahid/order-service
    └── api-gateway/     ← submodule → ArfaMujahid/api-gateway
```

How the pointer model works (the key idea)

The workspace doesn't store the service code. Each submodule entry is just a **commit hash** pointing at a specific state of the child repo. That gives you:

- Services that are independently deployable and versionable.
- Different teams owning different repos.
- A workspace that always reflects a **known-good combination** of service versions.

Updating a service is therefore two moves: *push the service repo*, then *bump the pointer in the workspace*. (Section 3 covers this in detail — it's the part people get wrong.)

One submodule is not ours

`aidlc-workflows/` points at the upstream **awslabs/aidlc-workflows** repo, not one of our repos. We pin it at a known commit and only move that pointer deliberately. We never push to it — we just track the version of the methodology we're using.

Cloning the right way

```
# --recurse-submodules pulls the service repos too
git clone --recurse-submodules git@github.com:ArfaMujahid/AISLCTesting.git

# Forgot the flag and services/ came up empty? Fix it with:
git submodule update --init --recursive
```

3. Working With Submodules (the how-to)

Every submodule change touches **two repos**: the *service repo* (where the code lives) and the *workspace repo* (which stores the pointer). Nothing automatically links them — you make two deliberate commits, and **order matters**.

The two-repo mental model

Where you commit	What you're committing	Where it goes
<code>services/user-service</code>	actual code changes	<code>ArfaMujahid/user-service</code>
<code>AISLCTesting</code> (workspace root)	the updated submodule pointer (a commit hash)	<code>ArfaMujahid/AISLCTesting</code>

The golden rule

Always push the service repo first, then bump the workspace pointer. If you reverse it, other developers pull a commit hash that doesn't exist on the remote yet — and their clone breaks.

STEP 1 — COMMIT & PUSH INSIDE THE SUBMODULE

```
cd services/user-service
git checkout main          # avoid detached HEAD — see the footgun below
# ... make your edits ...
git add .
```

```
git commit -m "your change"
git push origin main      # → pushes to ArfaMujahid/user-service
```

At this point your code is safe on the service remote, but the workspace still points at the *old* commit — it has no idea anything changed.

STEP 2 — BUMP THE POINTER IN THE WORKSPACE

```
cd ../../                # back to the workspace root
git add services/user-service  # stages the NEW commit hash, not the code
git commit -m "bump user-service to <short-hash>"
git push                  # → pushes to ArfaMujahid/AISLCTesting
```

That second commit contains no service code — just the moved pointer. Confirm with `git diff --cached` and you'll see `-Subproject commit abc123... / +Subproject commit def456...` .

The detached-HEAD footgun

Submodules check out a specific *commit*, not a branch, so by default you're on a "detached HEAD." If you commit there without first running `git checkout main` , your commit isn't on any branch and `git push` won't know where to send it — the work appears to vanish.

Always `git checkout main` inside the submodule before editing.

Common submodule footguns

Problem	What happens	Fix
Cloned without <code>--recurse-submodules</code>	<code>services/</code> folders are empty	<code>git submodule update --init --recursive</code>
Submodule on detached HEAD	commits seem to disappear	<code>cd services/x && git checkout main</code>
Bumped pointer before pushing service	others get a missing commit hash	always push the service repo first
Forgot to bump after pushing service	others stay on the old version	<code>git add services/x && git commit && git push</code>

Merge conflict on the pointer	confusing diff of two hashes	pick the newer hash, commit
Pull latest of every service at once	—	<code>git submodule update --remote</code>

4. Solution, Part 2 — What AI-DLC Is and Why

AI-DLC (AI-Driven Development Life Cycle) is an intelligent, adaptive software-development workflow from AWS Labs. It turns "ask the AI to build something" into a structured lifecycle that **maintains quality standards and keeps humans in control at every step**.

The central bargain:

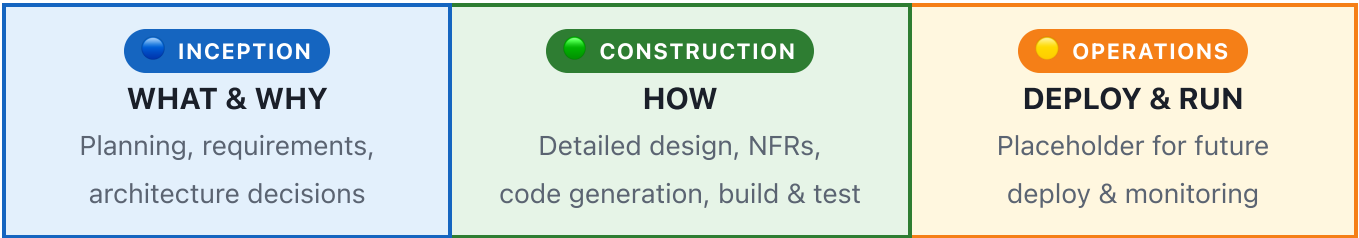
The AI does the work. Humans hold the gates. The AI analyses, plans, designs, and generates code — but it *stops at every stage boundary* and waits for explicit human approval before moving on. Every decision and every prompt is recorded.

What it gives you that raw prompting doesn't

- **Adaptive depth** — a one-line fix gets a light touch; a production service gets full requirements, design, and NFR treatment. The workflow adapts to the work, not the other way around.
- **Durable artifacts** — requirements, user stories, designs, and plans are written to files (`aidlc-docs/`), so decisions survive context resets and team handoffs.
- **Approval gates** — the AI never silently barrels ahead; you review and approve each stage.
- **A complete audit trail** — every prompt and response is logged with timestamps in `audit.md`.
- **A single source of truth** — design documents drive code generation, so the docs and the code stay in sync.

5. The Complete AI-DLC Flow

AI-DLC has a **three-phase lifecycle**. Each phase answers a different question.



The adaptive principle

Not every stage runs every time. Some stages are **ALWAYS** executed; the rest are **CONDITIONAL** — the AI decides whether they add value based on your intent, the existing codebase, and the complexity and risk of the change. There are *no fixed sequences*; stages run in the order that makes sense for the task.

The full flow, stage by stage

The diagram below shows the complete path. Green rows are always executed; amber rows run only when they add value. → is "next"; the Construction design stages form a **per-unit loop** that repeats for each unit of work before code generation.

Phase	Stage	Runs?	Purpose
INCEPTION what & why	Workspace Detection	Always	Scan the workspace; greenfield vs brownfield; resume if a prior session exists.
	Reverse Engineering	Conditional brownfield only	Analyse an existing codebase — architecture, components, APIs, dependencies.
	Requirements Analysis	Always adaptive depth	Capture intent; gather functional & non-functional requirements; ask clarifying questions.
	User Stories	Conditional	Create stories & personas when there's real user-facing impact.
	Workflow Planning	Always	Decide which stages to run and at what depth; visualise the plan.

	Application Design	Conditional	Identify components, methods, business rules, and service layers.
	Units Generation	Conditional	Decompose the system into independent units of work .
 CONSTRUCTION how — per-unit loop	Functional Design	Conditional per-unit	Business logic, domain models, data schemas — technology-agnostic.
	NFR Requirements	Conditional per-unit	Performance, security, scalability; select the tech stack.
	NFR Design	Conditional per-unit	Apply NFR patterns and logical components to the design.
	Infrastructure Design	Conditional per-unit	Map the design to actual cloud / infrastructure services.
	Code Generation	Always per-unit	Part 1: Plan (checkbox file list) → Part 2: Generate code & tests.
	Build & Test	Always after all units	Build instructions, unit / integration / performance test instructions.
 OPERATIONS	Operations	Placeholder	Reserved for future deploy, monitoring, and incident workflows.

The per-unit loop. In Construction, each unit of work is taken all the way through its design stages and Code Generation *before* the next unit starts. Once every unit is done, a single Build & Test stage closes out the whole effort.

Depth levels

Level	When	What you get
Minimal	simple, clear request	Just enough — document the intent and move.

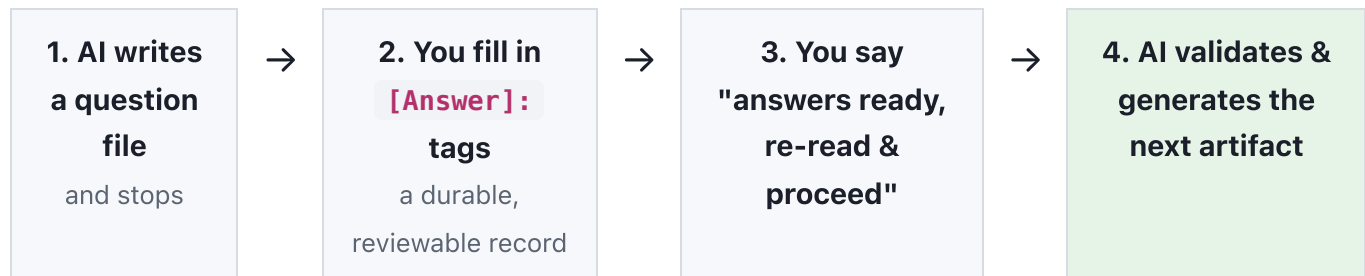
Standard	normal complexity	Functional + non-functional requirements, standard artifacts.
Comprehensive	complex / high-risk / production-critical	Full depth, full traceability, all artifacts.

You can influence it directly: *"Keep this at minimal depth"* or *"This is production-critical — run at comprehensive depth."*

6. How You Interact With It — the Gate Model

This is the part that feels different from normal AI chat. AI-DLC does not ask clarifying questions inline and then guess. It writes questions to a file, waits, and stops at every stage boundary.

The Question → Doc → Approval flow



Questions are multiple-choice. You answer in the file itself:

```

## Question: Deployment model
Where will this service be deployed?

A) AWS Lambda (serverless)
B) AWS ECS Fargate (containerized)
C) Existing on-premises infrastructure
X) Other (describe after the [Answer]: tag)

[Answer]: B – containerized; we already run ECS in this account
  
```

Tips that work well: add a label next to the letter, include a one-line justification, combine options when you mean both (**B and C**), and use **X** freely when nothing fits rather than forcing a wrong answer.

Approval gates

At the end of each stage, AI-DLC presents a standardized **two-option** completion message:

Request Changes	Approve & Continue
Ask for modifications before moving on. The AI revises and re-presents.	Accept the output and advance to the next stage.

Read the generated artifact before you approve. Design documents are the source of truth for code generation — mistakes that slip through a gate are much harder to fix later.

Context management — clear at the gates

Each gate is a natural place to **start a fresh context**. The AI's work is already saved to files, so a clean reset is low-risk: the next context loads the relevant artifacts from disk and continues without dragging along accumulated noise. If your tool offers a "compact context" prompt mid-workflow, decline it — compaction loses more than a clean reset. To resume:

```
Go to aidlc-docs/aidlc-state.md, find the first unchecked item,
then go to the corresponding plan file and resume from that point.
```

The two files that track everything

File	Role
aidlc-docs/aidlc-state.md	Progress & gate checklist — which stages ran, which were skipped, what's next.
aidlc-docs/audit.md	Complete audit trail — every prompt and response, raw and timestamped (ISO 8601). Always appended, never overwritten.

7. The Core Discipline — Never Vibe Code

"Vibe coding" means directly editing generated code to make a quick fix — bypassing the design documents. It feels fast and creates drift: the design docs (the source of truth AI-DLC reads for every subsequent operation) no longer match what the code actually does. The

next code-generation run, session resume, or teammate handoff then hits confusion and rework.

The rule: update the design first, then regenerate the code. Never fix the code directly.

The right way to make any change

Step	What you do
1	Describe the issue, touch nothing. <i>"Do not update any documents yet. I've found issue X. Help me understand where it needs to be addressed."</i>
2	Fix the design document. <i>"Update [design doc] to reflect the fix, then check whether requirements or user stories also need updating."</i>
3	Regenerate the affected code. <i>"The design for [unit] changed — re-run code generation for the affected files only."</i>

Two standing rules that prevent drift

Set these once at the start of Construction and they apply for the whole phase:

- **Back-propagation:** "Every time you update a document, check whether the change impacts the requirements document and user stories, and prompt me if it does."
- **Design-first:** "When you make a design decision during code generation, always make sure the documentation reflects it before proceeding."

One more practical note: ask exploratory questions safely by prefixing them with *"Do not update any documents."* — otherwise the AI may interpret a question as a change request and start editing.

8. How AI-DLC Is Wired Into This Repo

AI-DLC isn't copied into the workspace — it's pulled in as a submodule, the same mechanism as the services, so we track an exact version of the methodology.

Piece	What it is
-------	------------

<code>aidlc-workflows/</code>	Submodule → <code>aws-labs/aidlc-workflows</code> . The steering rules the AI follows. External and pinned — we never push to it; we only move the pointer deliberately to adopt a new version.
<code>aidlc-docs/</code>	The working artifacts AI-DLC produces (requirements, designs, state, audit). Tracked in git , so the decisions are versioned alongside the workspace.
<code>CLAUDE.md</code>	Global context the AI reads first — including the rule to follow AI-DLC and the cross-service rules.

Where code goes vs where docs go

Application code → the workspace root (inside the relevant service). **Documentation** → `aidlc-docs/` only. Never put code in `aidlc-docs/`, and keep human-facing reports out of it too — files there are reloaded as design input on later stages and would inflate context.

The artifacts AI-DLC writes

```
aidlc-docs/
├─ aidlc-state.md           # progress / gate checklist
├─ audit.md                 # full, timestamped audit trail
├─ inception/
│   ├─ requirements/        # requirement docs + verification questions
│   ├─ user-stories/
│   └─ application-design/
├─ construction/
│   ├─ {unit-name}/
│   │   ├─ functional-design/
│   │   ├─ nfr-requirements/
│   │   └─ nfr-design/
│   └─ code/                # markdown summaries (code itself lives at root)
├─ build-and-test/
└─ operations/              # placeholder
```

Kicking off a session

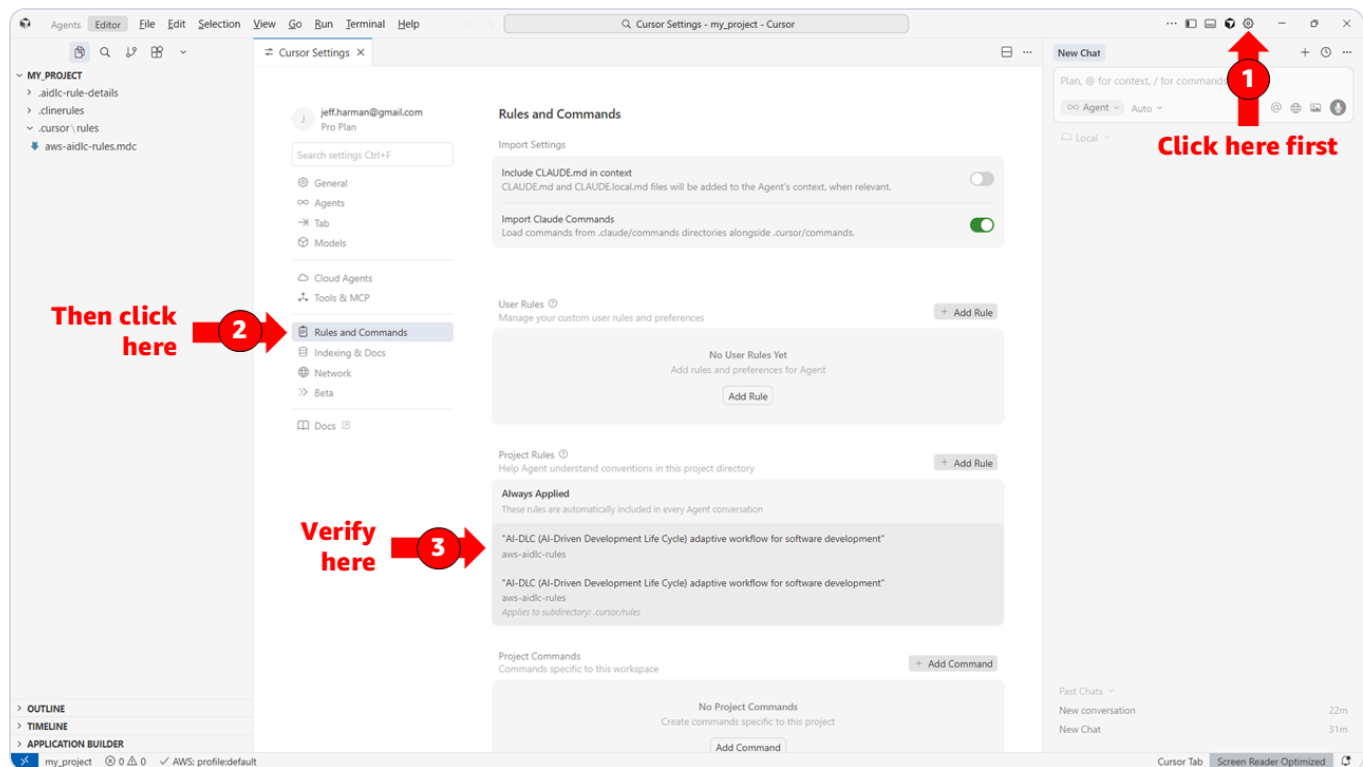
Prepare two short inputs first — they dramatically cut the number of clarifying questions:

- **Vision Document** — what you're building and why; MVP features in scope; what's explicitly out of scope; known open questions.
- **Technical Environment Document** — language/version, framework, cloud + deploy model, test framework, a *prohibited-libraries* table (with reason + alternative), and example code patterns for a typical endpoint, function, and test.

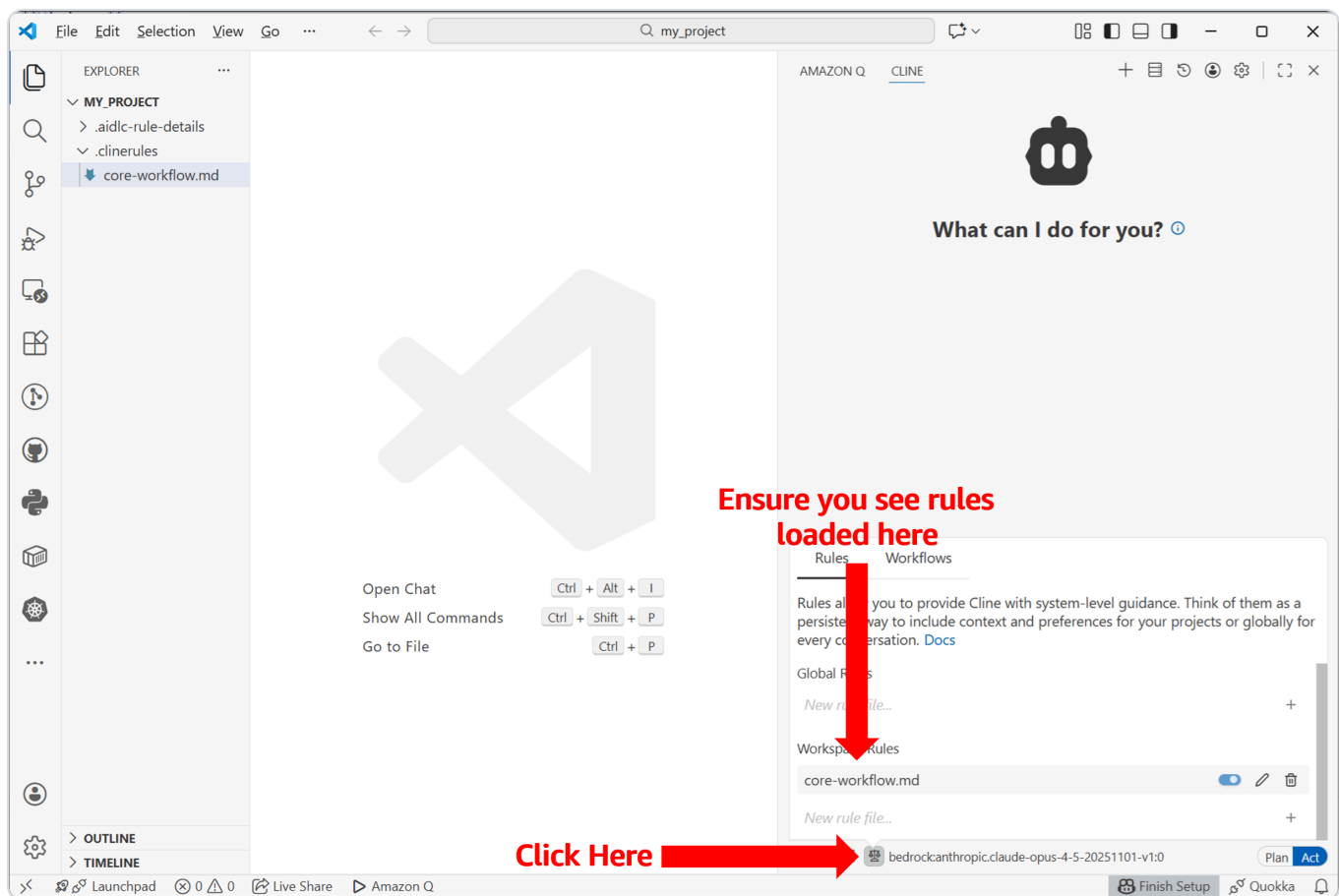
I want to start a new project. Please read [vision document] and [technical environment document], then begin the AI-DLC workflow.

9. AI-DLC in Your IDE

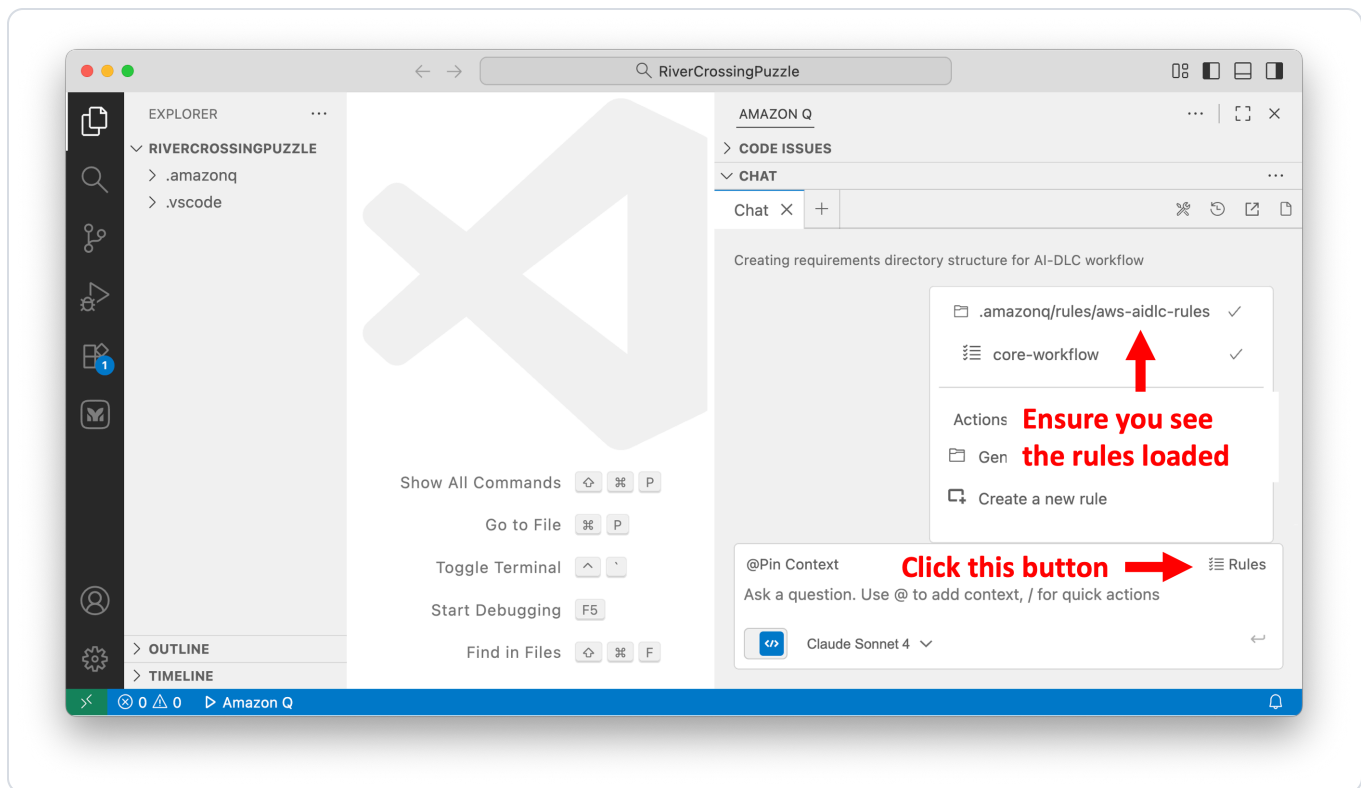
AI-DLC is tool-agnostic — the same rules load across coding assistants. Once the rules are in place, the assistant greets you with the AI-DLC workflow and starts gating its work. Below: the rules loaded in several environments.



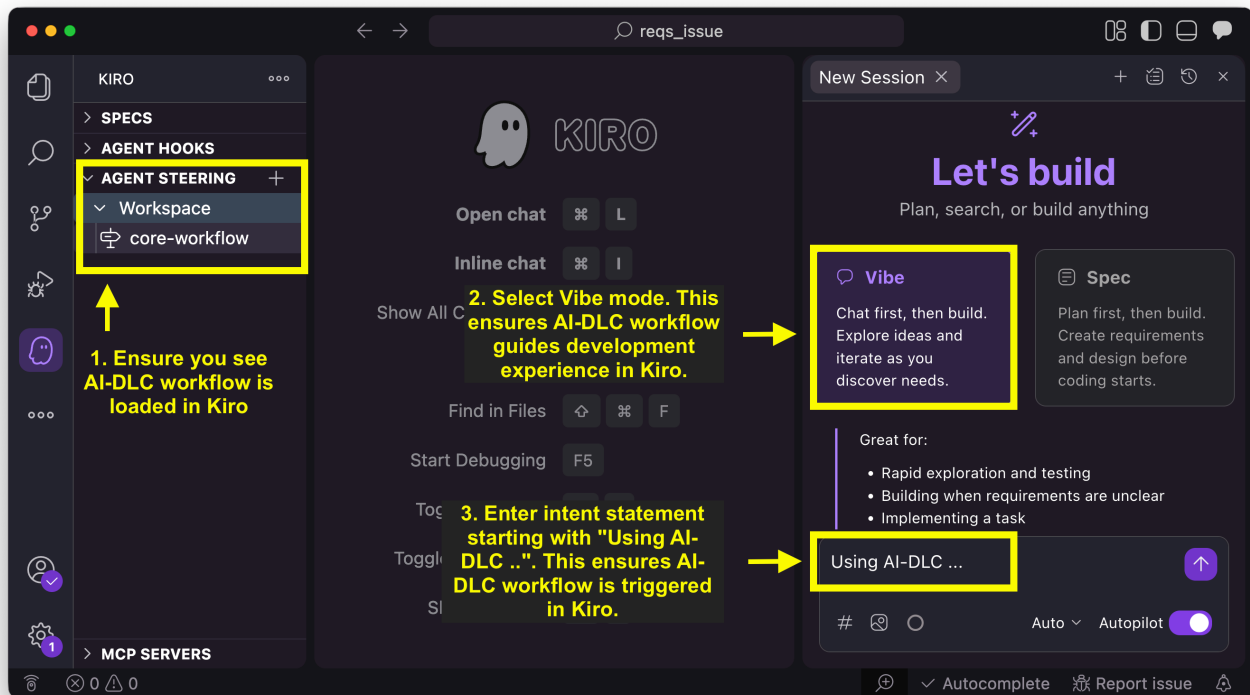
Cursor IDE — AI-DLC rules loaded as project rules.



Cline — AI-DLC rules recognised and active.



Amazon Q Developer — AI-DLC steering rules loaded.



Kiro IDE — AI-DLC rules loaded as Kiro steering files.

10. Glossary & Quick Reference

TERMINOLOGY

Term	Meaning
Phase	One of the three lifecycle phases: Inception, Construction, Operations.
Stage	An individual activity within a phase (e.g. Requirements Analysis). Either ALWAYS or CONDITIONAL.
Greenfield / Brownfield	A brand-new project vs. work added to an existing codebase (which triggers Reverse Engineering).
Unit of Work (UOW)	A logical grouping of stories taken through design + code as one chunk; often becomes a service.
NFR	Non-Functional Requirement — performance, security, scalability, etc.

Component / Module / Service	Reusable building block / logical grouping within a service / independently deployable unit.
Plan vs Artifact	A plan is a checkbox/question file that guides execution; an artifact is the generated output.

SUBMODULE COMMANDS

Goal	Command
Clone with services	<code>git clone --recurse-submodules <url></code>
Fix empty <code>services/</code>	<code>git submodule update --init --recursive</code>
Pull latest of all services	<code>git submodule update --remote</code>
Push a service change	<code>cd services/x && git checkout main && git commit -am "..."</code> <code>&& git push</code>
Bump the workspace pointer	<code>cd ../.. && git add services/x && git commit -m "bump x"</code> <code>&& git push</code>

THE NON-NEGOTIABLES

- Run the AI from the **workspace root**, never from inside a service folder — that's what gives it cross-service context.
- **Push the service repo before bumping** the workspace pointer.
- Update `shared-contracts/` **before** changing any API shape or event.
- **Never vibe code** — update the design, then regenerate.
- Read each artifact **before** approving a gate.

Sources: this workspace's `CLAUDE.md` and `README.md`, and the `aws-labs/aidlc-workflows` submodule (core workflow, process overview, terminology, and the "Working with AI-DLC" guide). Learn more: the AWS AI-DLC blog and Method Definition Paper linked from the upstream repo.